

Linux同步管理

原子操作

编写内核代码或驱动代码时需要留意共享资源的保护,防止共享资源被并发访问。所谓并发访问,是指多个内核路径同时访问和操作数据,有可能发生相互覆盖共享数据的情况,造成被访问数据的不一致。内核路径可以是一个内核执行路径、、中断处理程序或者内核线程等。

并发访问可能会造成系统不稳定或产生错误,且很难跟踪和调试。

在早期不支持SMP对称多处理器的Linux内核中,导致并发访问的因素是中断服务程序,只有在中断发生时,或者内核代码路径显式地要求重新调度并「且执行另一个进程时,才有可能发生并发访问。在支持SMP对称多处理器的Linux内核中,并发运行在不同CPU中的内核线程完全有可能在同一时刻并发访问共享数据,并发访问随时可能发生。特别是现在的Linux内核已经支持内核抢占,调度器可以抢占正在运行的进程,重新调度其他进程来执行。

在计算机术语中,临界区是指访问和操作共享数据的代码段,过这些资源无法同时被多个执行线程访问,访问临界区的执行线程或代码路径称为并发源。为了避免临界区中的并发访问,开发者必须保证访问临界区的原子性,也就是说在临界区内不能有多个并发源同时执行,整个临界区就像一个不可分割的整体。

```
static int i = 0;
```

```
void thread1_func(void) { i++; }
```

```
void thread2_func(void) { i++; }
```

时间	线程1操作	线程2操作	内存中的i
t1	读 i → 得到0		0
t2		读 i → 得到0 (线程2也读到0, 因为线程1还没写回)	0
t3	tmp = 0 + 1 tmp = 1		0
t4		tmp = 0 + 1 tmp = 1	0
t5	写 i = 1		1
t6		写 i = 1 (线程2把1写回去, 覆盖了线程1的结果)	1

结果: i = 1 (期望应该是 i = 2)

从上面代码的执行示意图来看,最终结果有可能等于1。因为变量i是一个临界资源,CPU0和CPU1都有可能同时访问它,从而发生并发访问。从CPU角度来看,变量i作为一个静态全局变量存储在数据段中,首先读取变量的值到通用寄存器中,然后在通用寄存器里做i++运算,最后把寄存器的数值写回变量i所在的内存中。

在多处理器架构中,上述动作有可能同时进行。如果线程B函数在某个中断处理函数中执行,在单处理器架构上仍可能会发生并发访问。

可以使用加锁的方式,例如用spinlock 来保证 i++操作的原子性,但是加锁操作导致比较大的开销,用在这里有些浪费。

Linux内核提供了atomic_t 类型的原子变量,它的实现依赖于不同的体系结构。atomic_t类型的具体定义如下:

```
// 原子变量版本 (正确)
static atomic_t j = ATOMIC_INIT(0);

void thread1_func(void) { atomic_inc(&j); }
void thread2_func(void) { atomic_inc(&j); }
```

内存屏障

x86 有 写缓冲区 (Store Buffer)，CPU 执行写操作时：

- 1. 数据先进入写缓冲区 (不直接写内存)
- 2. CPU 可以继续执行后续的读操作
- 3. 写缓冲区稍后才将数据刷新到内存

写-读会重排？

x86 有 写缓冲区 (Store Buffer)，CPU 执行写操作时：

- 1. 数据先进入写缓冲区 (不直接写内存)
- 2. CPU 可以继续执行后续的读操作
- 3. 写缓冲区稍后才将数据刷新到内存

程序顺序：	实际执行顺序（可能）：
Store A = 1	Store A 进入写缓冲区
Load B	Load B 执行（此时A还未写入内存）
Store A	从缓冲区写入内存

结果：Load B 在 Store A "之前" 执行了！

```
#define __mb()    asm volatile("mfence" ::: "memory")    // 全屏障
#define __rmb()   asm volatile("lfence" ::: "memory")    // 读屏障
#define __wmb()   asm volatile("sfence" ::: "memory")    // 写屏障
```

指令	全名	作用
mfence	Memory Fence	确保所有 Load/Store 有序
lfence	Load Fence	确保所有 Load 有序，阻塞后续指令
sfence	Store Fence	确保所有 Store 有序，刷新写缓冲区

SMP 内存屏障（多核间同步）

```

/* SMP 全屏障：使用 lock addl */
#define __smp_mb() asm volatile("lock addl $0,-4(%%_ASM_SP)" ::: "memory", "cc")

/* SMP 读屏障/
#define __smp_rmb() dma_rmb()

/* SMP 写屏障
#define __smp_wmb() barrier()

```

内存屏障

现代计算机系统中，内存操作的实际执行顺序可能与程序代码中的顺序不同。这是因为：内存屏障产生的原因：

1. 编译器优化：编译器可能重排指令以提高性能
2. CPU 优化：CPU 可能乱序执行指令、投机加载、合并写操作等
3. Cache 层级：多级缓存可能导致不同 CPU 看到不同顺序的内存更新

```

# define barrier() __asm__ __volatile__("" : : "memory") // 只影响编译阶段， 阻止编译器优化和重排，

```

单处理器内存屏障指令

```

#define __mb() asm volatile("mfence" ::: "memory") // Memory Fence (全屏障)
#define __rmb() asm volatile("lfence" ::: "memory") // Load Fence (读屏障)
#define __wmb() asm volatile("sfence" ::: "memory") // Store Fence (写屏障)

```

用于 CPU 与设备/DMA 之间的同步。

多核SMP环境下的内存屏障指令

```

#define __smp_mb()      asm volatile("lock addl $0,-4(%%" _ASM_SP ") " ::: "memory", "cc")
#define __smp_rmb()     dma_rmb()
#define __smp_wmb()     barrier()
``` key

```c
// CPU0
void func0() {
    a = 1;
    // smp_wmb
    b = 1;
}
// CPU1
void func1() {
    while (b == 0) continue;
    // smp_rmb
    assert(a == 1);
}

```

最后判断`assert(a == 1)`也可能失败

使用`__smp_wmb()`将当前存储缓冲区（Store Buffer）的所有数据做一个标记，然后写入到存储缓冲区，保证之前写入到存储缓冲区的数据更新到高速缓存行，然后再执行后面的写操作

`__smp_rmb()`读内存屏障可以让无效队列里的无效操作都执行完成才能执行读屏障指令后面的读操作。

自旋锁

原子变量能提供底层的锁机制，为什么还需要spinlock

如果临界区只是一个变量,那么原子变量可以解决问题。但是临界区大多是一个数据操作的集合,例如先从一个数据结构中移出数据,对其进行数据解析,再写回到该数据结构或者其他数据结构中,类似"read->modify->write"操作;再比如临界区是一个链表操作等。

整个执行过程需要保证原子性,在数据被更新完毕前,不能有其他内核代码路径访问和改写这些数据。这个过程使用原子变量显得不合适,需要锁机制来完成。自旋锁(spinlock)是Linux内核中最常见的锁机制。

自旋锁同一时刻只能被一个内核代码路径持有,如果有另一个内核代码路径试图获取一个已经被持有的自旋锁,那么该内核代码路径需要一直忙等待,直到锁持有者释放了该锁。如果该锁没有被别人持有(或争用),那么可以立即获得该锁。自旋锁的特性如下:

- 忙等待的锁机制。操作系统中锁的机制分为两类,一类是忙等待,另一类是睡眠等待。自旋锁属于前者,当无法获取自旋锁时会不断尝试,直到获取锁为止。
- 同一时刻只能有一个内核代码路径可以获得该锁。
- 要求自旋锁持有者尽快完成临界区的执行任务。如果临界区执行时间过长,在锁外面忙等待的CPU比较浪费,特别是自旋锁临界区里不能睡眠
- 自旋锁可以在中断上下文中使用。

信号量

```
/* Please don't access any members of this structure directly */
struct semaphore {
    raw_spinlock_t    lock;
    unsigned int      count;
    struct list_head   wait_list;

#ifdef CONFIG_DETECT_HUNG_TASK_BLOCKER
    unsigned long      last_holder;
#endif
};
```

这三种锁都是可睡眠的锁,不允许在中断处理程序或者中断下半部(如tasklet等)中使用。

在中断上下文中可以毫不犹豫地使用自旋锁。

如果临界区有睡眠、隐含睡眠的动作及内核接口函数,应避免选选择自旋锁。

信号量和mutex的选择:满足mutex约束条件的,尽量选择mutex。。

信号量和mutex都是不区分读者和写者的。如果要区分读者和写者,那么只能使用读写信号量

互斥体

- 同一时刻只有一个线程可以持有Mutex。
- 只有锁持有者可以解锁。不能在一个进程中持有Mutex,而在房外一个进程中释放它。因此Mutex不适合内核同用户空间复杂的为同步场景,信号量和读写信号量比较适合。
- 不允许递归地加锁和解锁。
- 当进程持有Mutex时,进程不可以退出。
- Mutex必须使用官方API来初始化。
- Mutex可以睡眠,所以不允许在中断处理程序或者中断下半部中使用,例如tasklet、定时器等。

互斥锁由struct mutex使用一个原子变量 (->owner) 来跟踪 它们生命周期内的锁状态。字段owner实际上包含的是指向当前锁所有者的 struct task_struct * 指针, 因此如果无人持有锁, 则它的值为空 (NULL)。 由于task_struct的指针至少按L1_CACHE_BYTES对齐, 低位 (3) 被用来存储额外的状态

（例如，等待者列表非空）。在其最基本的形式中，它还包括一个等待队列和 一个确保对其序列化访问的自旋锁。

```
41 context_lock_struct(mutex) {
42     atomic_long_t      owner;
43     raw_spinlock_t      wait_lock;
44 #ifdef CONFIG_MUTEX_SPIN_ON_OWNER
45     struct optimistic_spin_queue osq; /* Spinner MCS lock */
46 #endif
47     struct list_head      wait_list;
48 #ifdef CONFIG_DEBUG_MUTEXES
49     void                  *magic;
50 #endif
51 #ifdef CONFIG_DEBUG_LOCK_ALLOC
52     struct lockdep_map      dep_map;
53 #endif
54 };
```

准备获得一把自旋锁时，有三种可能经过的路径，取决于锁的状态：

- 快速路径：试图通过调用`cmpxchg()`修改锁的所有者为当前任务，以此原子化地 获取锁。这只在无竞争的情况下有效（`cmpxchg()`检查值是否为0，所以3个状态 比特必须为0）。如果锁处在竞争状态，代码进入下一个可能的路径。
- 中速路径：也就是乐观自旋，当锁的所有者正在运行并且没有其它优先级更高的 任务（`need_resched`，需要重新调度）准备运行时，当前任务试图自旋来获得 锁。原理是，如果锁的所有者正在运行，它很可能不久就会释放锁。互斥锁自旋体 使用MCS锁排队，这样只有一个自旋体可以竞争互斥锁。
 - MCS锁（由Mellor-Crummey和Scott提出）是一个简单的自旋锁，它具有一些 理想的特性，比如公平，以及每个CPU在试图获得锁时在一个本地变量上自旋。它避免了常见的“检测-设置”自旋锁实现导致的（CPU核间）缓存行回弹（`cacheline bouncing`）这种昂贵的开销。一个类MCS锁是为实现睡眠锁的 乐观自旋而专门定制的。这种定制MCS锁的一个重要特性是，它有一个额外的属性，当自旋体需要重新调度时，它们能够退出MCS自旋锁队列。这进一步有助于避免 以下场景：需要重新调度的MCS自旋体将继续自旋等待自旋体所有者，即将获得 MCS锁时却直接进入慢速路径。
- 慢速路径：最后的手段，如果仍然无法获得锁，该任务会被添加到等待队列中， 休眠直到被解锁路径唤醒。在通常情况下，它以`TASK_UNINTERRUPTIBLE`状态 阻塞。

```

static __always_inline bool __mutex_trylock_fast(struct mutex *lock)
{
    unsigned long curr = (unsigned long)current;
    unsigned long zero = 0UL;

    // 单次原子交换: owner 从 0 变为 current
    if (atomic_long_try_cmpxchg_acquire(&lock->owner, &zero, curr))
        return true;

    return false;
}

```

进入中速路径的条件:

```

static inline int mutex_can_spin_on_owner(struct mutex *lock)
{
    // 条件 1: 无更高优先级任务
    if (need_resched())
        return 0;

    // 条件 2: 持有者正在 CPU 上运行
    owner = __mutex_owner(lock);
    if (owner)
        return owner_on_cpu(owner);

    // owner == 0: 锁已释放, 可以快速 trylock
    return 1;
}

```



```

// 中速路径
static __always_inline bool
mutex_optimistic_spin(struct mutex *lock, ...)
{
    // 1. 检查是否值得自旋
    if (!mutex_can_spin_on_owner(lock))
        goto fail;

    // 2. 获取 OSQ 锁（避免多个自旋者竞争）
    if (!osq_lock(&lock->osq))
        goto fail;

    // 3. 自旋循环
    for (;;) {
        owner = __mutex_trylock_or_owner(lock);
        if (!owner)
            break; // 成功获取

        // 检查持有者是否还在运行
        if (!mutex_spin_on_owner(lock, owner, ...))
            goto fail_unlock;

        cpu_relax();
    }

    osq_unlock(&lock->osq);
    return true;
}

```

不自旋的条件：

- 有更高优先级任务待运行 (need_resched())
- 持有者已睡眠（不在 CPU 上）
- 持有者被抢占

慢速路径

```

static __always_inline int __sched
__mutex_lock_common(struct mutex *lock, unsigned int state, ...)
{
    struct mutex_waiter waiter;

    // 1. 获取 wait_lock
    raw_spin_lock_irqsave(&lock->wait_lock, flags);

    // 2. 加入等待队列
    waiter.task = current;
    __mutex_add_waiter(lock, &waiter, &lock->wait_list);

    // 3. 设置睡眠状态
    set_current_state(state); // TASK_UNINTERRUPTIBLE

    // 4. 等待循环
    for (;;) {
        if (__mutex_trylock(lock))
            goto acquired;

        // 释放 wait_lock, 睡眠
        raw_spin_unlock_irqrestore(&lock->wait_lock, flags);
        schedule_preempt_disabled(); // 上下文切换

        // 被唤醒后重新尝试
        set_current_state(state);
        if (__mutex_trylock_or_handoff(lock, first))
            break;

        raw_spin_lock_irqsave(&lock->wait_lock, flags);
    }

acquired:
    __mutex_remove_waiter(lock, &waiter);
    raw_spin_unlock_irqrestore(&lock->wait_lock, flags);
    return 0;
}

```

读写锁

读写锁通常允许多个线程并发地读访问临界区,但是写访问只限制于一个线程

特点:

- 允许多个读者同时进入临界区,但同一时刻写者不能进入
- 同一时刻只允许一个写者进入临界区。
- 读者和写者不能同时进入临界区。

```
13 typedef struct qrwlock {
14     union {
15         atomic_t cnts; // 原子变量
16         struct {
17 #ifdef __LITTLE_ENDIAN
18             u8 wlocked;      /* Locked for write? */
19             u8 __lstate[3];
20 #else
21             u8 __lstate[3];
22             u8 wlocked;      /* Locked for write? */
23 #endif
24         };
25     };
26     arch_spinlock_t    wait_lock;
27 } arch_rwlock_t;

29 #define __ARCH_RW_LOCK_UNLOCKED {          \
30     { .cnts = ATOMIC_INIT(0), },          \
31     .wait_lock = __ARCH_SPIN_LOCK_UNLOCKED, \
32 }
```

状态位:

```
27 #define _QW_WAITING    0x100      /* A writer is waiting */
28 #define _QW_LOCKED     0x0ff      /* A writer holds the lock */
29 #define _QW_WMASK      0x1ff      /* Writer mask */
30 #define _QR_SHIFT      9          /* Reader count shift */
31 #define _QR_BIAS       (1U << _QR_SHIFT)
```

原子变量cnt布局:

Bit 31 ... Bit 9	Bit 8	Bit 7 ... Bit 0
读者计数	写者等待	写者持有
(N × 0x200)	WAITING	LOCKED

cnts = 0x000 → 空闲（无读者、无写者）

cnts = 0x0ff → 写者持有

cnts = 0x100 → 写者等待，无读者

cnts = 0x200 → 1个读者

cnts = 0x400 → 2个读者

cnts = 0x600 → 3个读者

cnts = 0x700 → 写者等待 + 3个读者（cnts = 0x100 + 0x600）

读锁获取：

```
static inline void queued_read_lock(struct qrwlock *lock)
{
    int cnts;

    // 乐观增加读者计数
    cnts = atomic_add_return_acquire(_QR_BIAS, &lock->cnts);

    // 检查写者位（持有或等待）
    if (likely(!(cnts & _QW_WMASK)))
        return; // 成功

    // 有写者，进入慢路径
    queued_read_lock_slowpath(lock);
}
```

读锁获取（慢路径）

```

void queued_read_lock_slowpath(struct qrwlock *lock)
{
    // 中断上下文特殊处理：不排队，直接自旋
    if (unlikely(in_interrupt())) {
        atomic_cond_read_acquire(&lock->cnts, !(VAL & _QW_LOCKED));
        return;
    }

    // 撤销之前乐观增加的计数
    atomic_sub(_QR_BIAS, &lock->cnts);

    trace_contention_begin(lock, LCB_F_SPIN | LCB_F_READ);

    // 获取等待队列锁
    arch_spin_lock(&lock->wait_lock);

    // 在队列保护下再次增加计数
    atomic_add(_QR_BIAS, &lock->cnts);

    // 自旋等待写者释放
    atomic_cond_read_acquire(&lock->cnts, !(VAL & _QW_LOCKED));

    // 释放队列锁
    arch_spin_unlock(&lock->wait_lock);

    trace_contention_end(lock, 0);
}

```

写锁获取

```

static inline void queued_write_lock(struct qrwlock *lock)
{
    int cnts = 0;

    // 尝试从空闲状态直接获取
    if (likely(atomic_try_cmpxchg_acquire(&lock->cnts, &cnts, _QW_LOCKED)))
        return; // 成功

    queued_write_lock_slowpath(lock);
}

```

写锁获取（慢路径）

```

void queued_write_lock_slowpath(struct qrwlock *lock)
{
    int cnts;

    trace_contention_begin(lock, LCB_F_SPIN | LCB_F_WRITE);

    // 获取等待队列锁
    arch_spin_lock(&lock->wait_lock);

    // 尝试直接获取
    if (!(cnts = atomic_read(&lock->cnts)) &&
        atomic_try_cmpxchg_acquire(&lock->cnts, &cnts, _QW_LOCKED))
        goto unlock;

    // 设置等待标志, 阻止新读者
    atomic_or(_QW_WAITING, &lock->cnts);

    // 等待所有读者和写者退出
    // 目标: cnts 只剩 _QW_WAITING 位 = 1
    do {
        cnts = atomic_cond_read_relaxed(&lock->cnts, VAL == _QW_WAITING);
    } while (!atomic_try_cmpxchg_acquire(&lock->cnts, &cnts, _QW_LOCKED));

unlock:
    arch_spin_unlock(&lock->wait_lock);

    trace_contention_end(lock, 0);
}

```

RCU

Linux内核中已经有了原子操作、spinlock、读写spinlock、读写信号量mutex等锁机制,为什么要单独设计一个比它们实现要复杂得多的新机制呢?

RCU机制要实现的目标是: 希望读者线程没有同步开销,或者说同步开销变得很小,甚至可以忽略不计,不需要额外的锁,不需要使用原子操作指令和内存屏障,即可畅通无阻地访问,而把需要同步的任务交给写者线程,写者线程等待所有读者线程完成后才会把旧数据销毁

RCU记录所有指向共享数据的指针的使用者,当要修改该共享数据时,首先创建一个副本,在副本中修改。所有读访问线程都离开读临界区之后,指针指向新的修改后副本的指针,并且删除旧数据。

RCU经常用于读比写者高的场景,进入RCU保护的临界区后不能睡眠。

```

// 读者侧
rcu_read_lock()    // 进入读侧临界区
rcu_read_unlock()  // 退出读侧临界区
rcu_dereference()  // 安全读取 RCU 保护指针

// 写者侧（更新者）
rcu_assign_pointer() // 发布新指针（含内存屏障）
synchronize_rcu()    // 阻塞等待宽限期结束
call_rcu()           // 异步回调，宽限期结束后调用

```

```

static inline void __rcu_read_lock(void)
{
    preempt_disable(); // 禁止抢占
}

static inline void __rcu_read_unlock(void)
{
    ....
    preempt_enable(); // 恢复抢占
}

```

`rcu_read_lock()` 几乎零开销——只是禁用抢占，读者不需要获取任何锁

读者侧

```

struct foo __rcu *global_ptr; // __rcu 标记 RCU 保护指针

// 读侧临界区
rcu_read_lock();
p = rcu_dereference(global_ptr); // 安全获取指针
if (p) {
    value = p->field; // 使用数据
}
rcu_read_unlock();

```

写者侧

```

void update_foo(int new_value)
{
    struct foo *new_p, *old_p;

    // 准备新数据
    new_p = kmalloc(sizeof(*new_p), GFP_KERNEL);
    new_p->field = new_value;

    // 保护写侧并发
    spin_lock(&lock);
    old_p = rcu_dereference_protected(global_ptr, lockdep_is_held(&lock));

    // 发布新指针（移除阶段）
    rcu_assign_pointer(global_ptr, new_p);
    spin_unlock(&lock);

    // 等待宽限期（回收阶段）
    synchronize_rcu(); // 阻塞直到所有读者完成

    // 安全释放旧数据
    kfree(old_p);
}

```

写者侧 - 异步方式（call_rcu）


```

struct foo {
    int field;
    struct rcu_head rcu; // 必须内嵌 rcu_head
};

// 回调函数
static void foo_reclaim(struct rcu_head *rh)
{
    struct foo *p = container_of(rh, struct foo, rcu);
    kfree(p);
}

void update_foo_async(int new_value)
{
    struct foo *new_p, *old_p;

    new_p = kmalloc(sizeof(*new_p), GFP_KERNEL);
    new_p->field = new_value;

    spin_lock(&lock);
    old_p = rcu_dereference_protected(global_ptr, lockdep_is_held(&lock));
    rcu_assign_pointer(global_ptr, new_p);
    spin_unlock(&lock);

    // 异步回收：宽限期结束后自动调用回调
    call_rcu(&old_p->rcu, foo_reclaim);

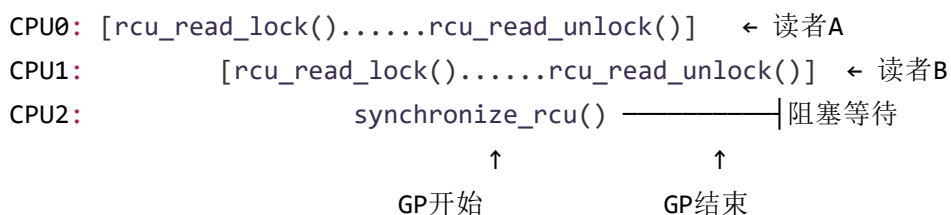
    // 函数立即返回，不阻塞
}

```

Grace Period（宽限期）机制

宽限期是从更新发生到所有可能持有旧数据引用的读者完成的时间窗口。

时间线：



synchronize_rcu() 只等待已经开始的读侧临界区，不等待之后新开始的。

RCU 如何知道读者完成了？通过检测Quiescent State（静默状态）

```
struct rcu_data {
    // 每CPU数据
    unsigned long gp_seq;           // 当前宽限期序列号
    bool core_needs_qs;           // 需要报告静默状态
    union rcu_noqs cpu_no_qs;      // 还未报告静默状态
    struct rcu_node *mynode;       // 该CPU对应的叶子节点
    // ...
};
```

静默状态的判定：

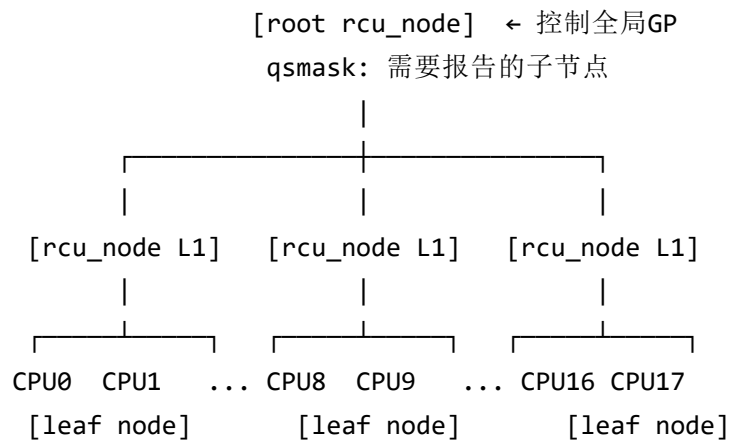
- 进程上下文切换
- idle 循环
- ...

只要发生上下文切换，就必定已退出所有读侧临界区（因为读侧临界区不能阻塞）。

```
/*
 * RCU 使用树形层次结构跟踪所有CPU的静默状态
 */

struct rcu_node {
    raw_spinlock_t lock;
    unsigned long gp_seq;           // 宽限期序列号
    unsigned long qsmask;           // 需要报告QS的CPU/子节点掩码
    int grplo, grphi;              // 该节点覆盖的CPU范围
    struct rcu_node *parent;
    // ...
};

struct rcu_state {
    struct rcu_node node[NUM_RCU_NODES];
    struct rcu_node *level[RCU_NUM_LVL+1]; // 各层指针
    unsigned long gp_seq;           // 全局宽限期序列号
    struct task_struct *gp_kthread; // GP内核线程
    // ...
};
```



每个叶子节点覆盖一组CPU。静默状态报告从叶子向上传播，当根节点的 qsmask 清零，宽限期结束。

宽限期检测流程：

1. GP内核线程启动新宽限期

- 设置 root->gp_seq 新值
- 设置各节点的 qsmask = qsmaskinit (需要报告的CPU)

2. 各CPU检测静默状态

- 在时钟tick、上下文切换等点检查
- 发现自己已处于QS，向叶子rcu_node报告
- 清除 qsmask 中对应的位

3. QS向上传播

- 叶子节点qsmask清零 → 向父节点报告
- 父节点清除对应位
- 直到根节点qsmask清零

4. 宽限期完成

- 唤醒等待的 synchronize_rcu() 调用者
- 执行 call_rcu() 注册的回调

上下文切换时候会标记静默状态

```

void rcu_note_context_switch(bool preempt)
{
    struct rcu_data *rdp = this_cpu_ptr(&rcu_data);

    // 标记本 CPU 已处于静默状态
    rcu_qs();

    if (rdp->cpu_no_qs.b.exp)
        rcu_report_exp_rdp(rdp);
}

static void rcu_qs(void)
{
    // 标记：本 CPU 已经处于静默状态
    if (__this_cpu_read(rcu_data.cpu_no_qs.b.norm)) {
        __this_cpu_write(rcu_data.cpu_no_qs.b.norm, false); // 清除标志
    }
}

```

每个时钟中断调用：

```

void rcu_sched_clock_irq(int user)
{
    // 检查静默状态并触发软中断
    rcu_flavor_sched_clock_irq(user);

    // 如果需要，触发 RCU 软中断
    if (rcu_pending(user))
        invoke_rcu_core(); // 触发 RCU_SOFTIRQ
}

// 非抢占版本
static void rcu_flavor_sched_clock_irq(int user)
{
    // 如果是用户态或 idle，说明已处于静默状态
    if (user || rcu_is_cpu_rrupt_from_idle()) {
        rcu_qs(); // 标记静默状态
    }
}

```

RCU软中断处理

```

static __latent_entropy void rcu_core(void)
{
    struct rcu_data *rdp = raw_cpu_ptr(&rcu_data);

    // 检查并报告静默状态
    rcu_check_quiescent_state(rdp);

    // 执行已完成的回调
    if (rcu_segcblist_ready_cbs(&rdp->cblist))
        rcu_do_batch(rdp);
}

static void rcu_check_quiescent_state(struct rcu_data *rdp)
{
    // 检查是否有新宽限期
    note_gp_changes(rdp);

    // 1. 本 CPU 是否需要报告静默状态？
    if (!rdp->core_needs_qs)
        return; // 不需要

    // 2. 本 CPU 是否已处于静默状态？
    if (rdp->cpu_no_qs.b.norm)
        return; // 还没处于静默状态

    // 3. 向 rcu_node 报告
    rcu_report_qs_rdp(rdp);
}

```

- core_needs_qs = true: 宽限期开始时设置，表示本 CPU 需要报告
- cpu_no_qs.b.norm = false: 表示本 CPU 已处于静默状态（由 rcu_qs() 设置）

叶子节点报告：

```

static void rcu_report_qs_rdp(struct rcu_data *rdp)
{
    unsigned long flags;
    unsigned long mask;
    struct rcu_node *rnp;

    rnp = rdp->mynode; // 本 CPU 对应的叶子节点

    raw_spin_lock_irqsave_rcu_node(rnp, flags);

    // 获取本 CPU 在 qsmask 中的位
    mask = rdp->grpmask;

    // 清除"需要报告"标志
    rdp->core_needs_qs = false;

    // 如果该位已经被清除，直接返回
    if ((rnp->qsmask & mask) == 0) {
        raw_spin_unlock_irqrestore_rcu_node(rnp, flags);
        return;
    }

    // 向 rcu_node 层次结构报告（向上传播）
    rcu_report_qs_rnp(mask, rnp, rnp->gp_seq, flags);
}

```

```

static void rcu_report_qs_rnp(unsigned long mask, struct rcu_node *rnp,
                             unsigned long gps, unsigned long flags)
{
    __releases(rnp->lock)
{
    // 向上遍历 rcu_node 树
    for (;;) {
        // 检查：该位是否已被清除？或宽限期已结束？
        if ((!(rnp->qsmask & mask) && mask) || rnp->gp_seq != gps) {
            raw_spin_unlock_irqrestore_rcu_node(rnp, flags);
            return; // 已经处理过了，直接返回
        }

        // 清除 qsmask 对应位
        WRITE_ONCE(rnp->qsmask, rnp->qsmask & ~mask);

        // qsmask 还有其他位未清除？
        if (rnp->qsmask != 0) {
            // 还有其他 CPU 未报告，返回
            raw_spin_unlock_irqrestore_rcu_node(rnp, flags);
            return;
        }

        // qsmask 清零！本节点所有 CPU 都报告了
        rnp->completedqs = rnp->gp_seq;

        // 向父节点报告
        mask = rnp->grpmask; // 本节点在父节点 qsmask 中的位

        if (rnp->parent == NULL) {
            // 已经到达根节点
            break; // 退出循环，持有根节点锁
        }

        // 释放当前节点锁，获取父节点锁
        raw_spin_unlock_irqrestore_rcu_node(rnp, flags);
        rnp = rnp->parent;
        raw_spin_lock_irqsave_rcu_node(rnp, flags);
    }

    // 根节点 qsmask 清零，宽限期完成
    rcu_report_qs_rsp(flags); // 唤醒 GP 内核线程
}

```

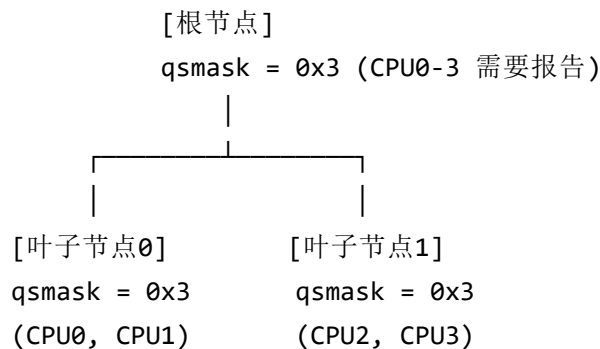
根节点处理，唤醒 GP 线程

```
static void rcu_report_qs_rsp(unsigned long flags)
    __releases(rcu_get_root()->lock)
{
    // 设置标志，通知 GP 内核线程
    WRITE_ONCE(rcu_state.gp_flags, rcu_state.gp_flags | RCU_GP_FLAG_FQS);

    raw_spin_unlock_irqrestore_rcu_node(rcu_get_root(), flags);

    // 唤醒 GP 内核线程处理宽限期结束
    rcu_gp_kthread_wake();
}
```


假设有 4 个 CPU，树形结构如下：



CPU0 发生上下文切换：

```
rcu_report_qs_rnp(mask=0x1, 叶子节点0):
    qsmask = 0x3 & ~0x1 = 0x2 (清除 CPU0 位)
    qsmask != 0 → return (CPU1 还没报告)
```

CPU1 发生上下文切换：

```
rcu_report_qs_rnp(mask=0x2, 叶子节点0):
    qsmask = 0x2 & ~0x2 = 0x0 (清除 CPU1 位)
    qsmask == 0 → 向父节点报告

获取根节点锁：
mask = 叶子节点0 在根节点的位 = 0x1
qsmask = 0x3 & ~0x1 = 0x2 (清除叶子节点0位)
qsmask != 0 → return (叶子节点1 还没报告)
```

CPU2 发生上下文切换：

```
rcu_report_qs_rnp(mask=0x1, 叶子节点1):
    qsmask = 0x3 & ~0x1 = 0x2 (清除 CPU2 位)
    qsmask != 0 → return (CPU3 还没报告)
```

CPU3 发生上下文切换：

`rcu_report_qs_rnp(mask=0x2, 叶子节点1):`

`qsmask = 0x2 & ~0x2 = 0x0` (清除 CPU3 位)

`qsmask == 0` → 向父节点报告

获取根节点锁:

`mask = 叶子节点1 在根节点的位 = 0x2`

`qsmask = 0x2 & ~0x2 = 0x0` (清除叶子节点1位)

`qsmask == 0` → 到达根节点且清零

`rcu_report_qs_rsp():`

唤醒 GP 内核线程 → 宽限期结束!

qsmask 是关键: 每个 `rcu_node` 有一个 `qsmask`, 每个位代表一个 CPU (叶子节点) 或一个子节点 (中间节点)。清除位的时机: 当 CPU 报告静默状态时, 清除叶子节点的对应位。

向上传播条件: 当叶子节点的 `qsmask` 清零, 向父节点报告, 清除父节点的对应位。

宽限期结束条件: 根节点的 `qsmask` 清零 → 所有 CPU 都报告了静默状态 → 宽限期结束